

Functions - User Defined, Types of arguments, Local and Global Namespace

User-Defined Functions in Python:

In Python, a user-defined function is created by the `def` keyword. Functions are blocks of reusable code designed to perform a specific task. They enhance code modularity and maintainability.

Syntax:

python

```
def function_name(parameters):  
    # Function body  
    # Code to perform the task  
    return result # Optional, as not all functions need to return a value
```

Example:

```
def greet(name):  
    """This function greets the person passed as a parameter."""  
    print(f"Hello, {name}!")  
  
# Calling the function  
greet("John")
```

Parameters:

- Parameters are variables used in the function definition.
- They act as **placeholders** for values that will be passed to the function when it is called.

Return Statement:

- The `return` statement is used to specify the result that the function should return.
- It is optional; if omitted, the function returns `None` by default.

Docstring:

- A docstring is a string literal that occurs as the first statement in a module, class, function, or method definition.
- It is used to document the purpose and usage of the function.

Scope:

- Variables defined inside a function have local scope and are only accessible within that function.

→ Local Scope

A variable created inside a function belongs to the *local scope* of that function, and can only be used inside that function.

```
def myfunc():
```

```
    x = 300
```

```
    print(x)
```

```
myfunc()
```

→ global scope.

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

- Variables defined outside any function have global scope and can be accessed throughout the entire program.

```
x = 300

def myfunc():

    print(x)

myfunc()

print(x)
```

Example with Return Statement:

```
def add_numbers(x, y):
    """This function adds two numbers and returns the result."""
    result = x + y
    return result

# Calling the function and storing the result
sum_result = add_numbers(5, 7)
print("The sum is",sum_result)
```

User-defined functions play a crucial role in structuring Python code and promoting code reusability. They allow developers to break down complex tasks into smaller, manageable functions.

Arguments in python

1. Positional Arguments:

- The most common type of argument.
- Values passed to the function based on their position in the function call.
- The order of arguments matters.

```
def example_function(arg1, arg2):  
    # Function body
```

```
example_function(value1, value2)
```

2. Keyword Arguments:

- Arguments passed to a function using the parameter names.
- The order of arguments is not important.

python

Copy code

```
def example_function(arg1, arg2):  
    # Function body
```

```
example_function(arg1=value1, arg2=value2)
```

3. Default Arguments:

- Parameters with default values assigned.

- If a value for the parameter is not provided during the function call, the default value is used.

```
def example_function(arg1, arg2=default_value):
    # Function body
    le_function(value1) # arg2 takes default_value
```

4. Variable-Length Arguments:

- Allow a function to accept any number of arguments.
- Denoted by `*args` for variable positional arguments

```
def print_values(*args):
    for value in args:
        print("my name is",value)

# Calling the function with different values
print_values(10, "Hello", 3.14)
```

```
my name is 10
my name is Hello
my name is 3.14
```

Functions: Recursions, Lambda, Filter, Map

→ Recursion:

Recursion is a programming concept where a function calls itself in order to solve a problem. In Python, a recursive function consists of a base case (to terminate the recursion) and a recursive case (where the function calls itself with a modified input).

```
def factorial1(x):  
    fact=1  
    for i in range(1,x+1):  
        fact=fact*i  
    print(fact)
```

```
factorial1(5)
```

120

using recursion

```
def fact2(n):  
    result = 1  
    if n > 1:  
        result = n * fact2(n - 1)  
    return result
```

```
fact2(5)
```

120

LAMBDA function

1. Anonymous Functions:

Lambda functions are also known as **anonymous functions** because they don't have a name. They are defined using the **lambda** keyword.

2. The syntax

of a lambda function is: **lambda arguments : expression.**

```
(lambda x, y: x + y)(3, 5)
```

8

3. Single Expression:

Lambda functions are limited to a single expression. They are ideal for short, simple operations where a full function definition might be too verbose.

Note:

```
x=8
if x%2==0:
    print("even")
else:
    print("odd")
```

even

```
x=66
"even" if x%2==0 else "odd"
```

'even'

```
(lambda x: "even" if x % 2 == 0 else "odd")(4)
```

'even'

```
add= lambda x, y: x + y
add(2,4)
```

6

Use with Higher-Order Functions:

Lambda functions are often used as arguments to higher-order functions like `map()`, `filter()`

1. `map()` Function:

- The `map()` function applies a given function to all items in an iterable (e.g., a list) and returns an iterator with the results.
- **Syntax:** `map(function, iterable)`
- **Example:**

```
numbers = [1, 2, 3, 4, 5]
sq= map(lambda x: x**2, numbers)
print(list(sq))
```

[1, 4, 9, 16, 25]

2. `filter()` Function:

- The `filter()` function constructs an iterator from elements of an iterable for which a function returns true.
- **Syntax:** `filter(function, iterable)`
- **Example:**

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even = filter(lambda x: x % 2 == 0, numbers)
print(list(even))
```

[2, 4, 6, 8, 10]